```
    for j in {0, .., |centroids|-1}
do                                          #5
      d ← distance(points[i],
centroids[j]                              #6
      if d < minDistance then
                           #7
        minDistance ←
d                                                    #8
        clusters[i] ← j
  return clusters
                                                          #9
```

#1 Method `classifyPoints` takes the list of points in the dataset and the current list of centroids. It returns the list of the centroids (technically, their indices) associated with each point in the dataset.
#2 Initialize the list of cluster assignments.
#3 Cycle through the list of points in the dataset (their indices).
#4 Initialize the minimum distance (between `points[i]` and any centroid) to the maximum possible value.
#5 Cycle through the list of centroids (again, their indices).
#6 Compute the distance between current point and centroid, and store it in a temporary variable.
#7 Check if the distance computed is smaller than the minimum found so far.
#8 If it is, update `minDistance` and assign the `i`-th point to the `j`-th centroid.
#9 Return the list of clusters.

We can move to the classification step, described in listing 12.3; method `classifyPoints` is a brute force search of all the point-centroid pairs, whose goal is finding the closest